

2º Relatório - Banco de Dados

Grupo: Gabriel Lorenzo Xavier, Jennifer Freire da Costa Silva, Luis Eduardo Pereira Nunes da Costa, Thiago Sergio Lima de Oliveira

Discente: Marcelo Iury de Sousa Oliveira

Introdução

O presente relatório descreve as principais decisões de implementação adotadas durante o desenvolvimento da Etapa 2 do projeto de Sistema de Gestão Hospitalar. São apresentadas as modificações realizadas no banco de dados, as justificativas para a utilização de Stored Procedures, Triggers e Views, bem como a escolha da ORM e das tecnologias empregadas no desenvolvimento da aplicação.

Alterações no Modelo de Dados

Na Etapa 1 do projeto, implementamos o modelo relacional básico do Sistema de Gestão Hospitalar, contemplando as entidades principais (Pessoa, Paciente, Profissional, Atendimento, Procedimento, etc.). No entanto, para atender aos novos requisitos da Etapa 2 (Triggers, Views e Stored Procedures), foi necessário realizar alterações estruturais no banco de dados. A seguir, detalhamos cada modificação e sua justificativa.

1. Adição da coluna “id_unidade” na tabela “Atendimento”: necessária para calcular o tempo médio de espera por unidade e para agrupar estatísticas por unidade.
2. Adição da coluna “data_hora_inicio” na tabela “Procedimento_Realizado”: necessária para calcular o tempo entre a chegada do paciente e o início do primeiro procedimento. Há tempos personalizados por tipo de atendimento para simular um atendimento real.
3. Adição da coluna “media_tempo_procedimento” na tabela “Procedimento”: necessária para armazenar a média real calculada a partir dos atendimentos realizados.
4. Criação da tabela “Auditoria_Atendimento”: necessária para rastrear as alterações nos atendimentos, registrando quem fez a alteração, quando e o que foi alterado. O uso de JSONB permite armazenar a estrutura completa do registro e foi exigido pelos requisitos do projeto.
5. Criação da tabela “Internação”: necessária para controlar as internações dos pacientes no hospital.
6. Distribuição dos atendimentos por unidade: como a coluna “id_unidade” foi criada após os atendimentos existirem, foi necessário preenchê-la com valores válidos, assim, os 10 atendimentos foram distribuídos entre as 4 unidades.

Stored Procedures

Em relação às Stored Procedures, segue a descrição de implementação de cada uma:

1. **sp_registrar_atendimento_completo:** Dentro do corpo da procedure, declaramos outro corpo (BEGIN [...] END) para que, assim, seja possível tratar qualquer possível erro que vier a aparecer no meio do registro. Por conta disso, o código contém um tratamento de erros em que, quando ele é acionado, o PostgreSQL nativamente recebe esse erro, aborta a transação e desfaz por completo tudo o que foi feito dentro da procedure. Além disso, o script finaliza inserindo os dados de atendimento na tabela Atendimento e, a partir de um JSON, lê cada procedimento realizado para também adicioná-los em sua respectiva tabela.
2. **sp_calcular_tempo_medio_espera:** Fizemos o uso de um Reference Cursor para pegar o resultado final da procedure e então mostrar ao usuário. Ele serve como um ponteiro para a consulta realizada

dentro da procedure, pois assim ela não precisa carregar tudo para a memória do servidor de uma só vez. Além disso, os dados são agrupados por Unidade e são feitas junções com as tabelas necessárias, inclusive por meio de uma consulta aninhada que obtém a menor `data_hora_inicio` entre os procedimentos realizados. No fim, o tempo médio é calculado, arredondado e, então, retornado junto aos nomes e IDs das unidades.

Um fato interessante é que, como essa procedure usa um cursor para apontar para uma consulta e, consequentemente, permite o acesso às suas linhas, seria possível, também, criar uma **função** no lugar dela, pois assim ela seria capaz de retornar uma tabela inteira, como um `SELECT` normalmente faz.

3. **sp_reajustar_escala:** Primeiro, foram realizadas as partes das verificações, que incluem verificar se a escala original realmente existe para o residente, ou seja, se o residente realmente está escalado na escala a ser trocada, e também verificar se a nova escala vai gerar conflito, ou seja, se o residente já estiver escalado para a mesma unidade, dia e turno.

Triggers VS Procedures

A escolha entre Triggers e Stored Procedures foi realizada considerando a finalidade de cada recurso dentro do PostgreSQL. Para a Etapa 2, optou-se pelo uso de Triggers em vez de Stored Procedures para as tarefas de validação, auditoria e manutenção de médias. Essa decisão foi baseada nos seguintes critérios:

- Triggers são executadas automaticamente em resposta a eventos (`INSERT`, `UPDATE`, `DELETE`), garantindo que as regras de negócio sejam aplicadas independentemente da origem da operação (aplicação, consulta manual, etc.). São ideais para validações em tempo real e auditoria, onde a ação deve ser imediata e obrigatória.
- Stored Procedures, por outro lado, exigem chamada explícita e são mais adequadas para operações complexas que envolvem múltiplos passos, transações e lógica condicional extensa. No projeto, as procedures foram utilizadas para operações como `sp_registrar_atendimento_completo`, que insere um atendimento e seus procedimentos em uma única transação.

Resumo da decisão: Triggers foram escolhidas para regras que devem ser aplicadas sempre e automaticamente, enquanto Procedures foram utilizadas para operações que exigem controle transacional e lógica complexa acionada sob demanda.

Triggers

Para a Etapa 2 do projeto, foram implementadas três triggers com o objetivo de automatizar regras de negócio, manter a integridade dos dados e registrar informações de auditoria.

1. **trg_check_sobreposicao_escala:** implementada na tabela **Escala**, é executada antes das operações de inserção e atualização. Sua finalidade é impedir que um mesmo residente seja escalado em duas unidades diferentes no mesmo dia e turno. Caso seja identificada uma sobreposição de escala, a operação é cancelada por meio de uma exceção.

2. **trg_audita_atendimento:** implementada na tabela **Atendimento**, é executada após operações de inserção, atualização e exclusão. Seu objetivo é registrar automaticamente essas operações na tabela **Auditoria_Atendimento**, armazenando o tipo da operação, o usuário, a data e hora da alteração e os dados do registro em formato **JSONB**. A tabela de auditoria não possui chave estrangeira para **Atendimento**, permitindo preservar o histórico mesmo após a exclusão do registro original.

3. trg_atualiza_media_procedimentos: implementada na tabela **Procedimento_Realizado**, é executada após a inserção de um procedimento realizado. Sua função é recalcular automaticamente a média do tempo real de execução do procedimento e atualizar a coluna **media_tempo_procedimento** da tabela **Procedimento**, mantendo esse valor sempre consistente.

As triggers implementadas centralizam regras de negócio no banco de dados, automatizando validações, auditorias e atualizações de informações, o que contribui para a integridade e a confiabilidade dos dados armazenados.

Views

Para a Etapa 2 do projeto, foram implementadas três views com o objetivo de simplificar consultas recorrentes e fornecer informações consolidadas para análise dos dados.

1. vw_pacientes_internados: reúne informações sobre os pacientes que permanecem internados no hospital, apresentando seus dados, a unidade de internação e a data de entrada. A consulta considera apenas internações cuja data de saída ainda não foi registrada, facilitando o acompanhamento dos pacientes atualmente internados.

2. vw_residentes_sem_supervisor: identifica os residentes escalados cujos preceptores não possuem titulação de Doutor. A view auxilia na verificação do cumprimento dos critérios de supervisão estabelecidos para os residentes, permitindo identificar rapidamente situações que podem exigir ajustes na escala.

3. vw_estatisticas_atendimentos_mensal: apresenta estatísticas dos atendimentos agrupadas por mês e unidade, incluindo a quantidade de atendimentos realizados, a duração média dos atendimentos e o procedimento mais frequente no período. Para identificar o procedimento mais realizado, foi utilizada a função analítica **RANK()**, permitindo exibir todos os procedimentos empatados na primeira posição.

As views implementadas simplificam consultas recorrentes, reduzem a repetição de comandos SQL e disponibilizam informações consolidadas para análise e apoio à gestão do sistema hospitalar.

ORM

- **Escolha da ORM**

Escolhemos inteiramente baseados na recomendação do professor e pela nossa maior familiaridade com a linguagem Python.

- **Reimplementação das operações da Etapa 1**

Reescrevemos o CRUD e as consultas da Etapa 1 usando a DSL do SQLAlchemy, sem SQL cru, como o enunciado pedia. A principal dificuldade foi a consulta de plantões por mês, que na versão em SQL usa *generate_series* — recurso do PostgreSQL sem equivalente direto na DSL do ORM. A solução foi calcular em Python quantas vezes cada dia da semana ocorre no mês corrente, deixando o join e o agrupamento por conta do SQLAlchemy.

- **Consultas avançadas**

Implementamos as três consultas exigidas pela Etapa 2: preceptores que supervisionaram residentes em atendimentos a pacientes flamenguistas, último atendimento de cada paciente e percentual de procedimentos de alto risco por residente.

A consulta de preceptores exigiu um alias para a tabela Pessoa, já que ela aparece duas vezes na mesma consulta — uma representando o preceptor, outra o paciente. A do último atendimento usa uma subquery para localizar a data mais recente por paciente, que depois é usada para recuperar a linha completa do atendimento correspondente; os procedimentos são buscados separadamente, para não duplicar as demais colunas no join principal. A de percentual soma os procedimentos por residente usando *case* para isolar, na mesma consulta, os que são de risco alto do total geral; o percentual em si é calculado em Python a partir desses dois totais.

- **Tratamento de concorrência e transações**

Para esta etapa, foram considerados os dois cenários: otimista e pessimista. Ambas foram implementadas por conta do fato do cenário otimista já estar implicitamente programado, não necessitando de um tratamento complexo (faltando, é claro, a paralelização). Para ambos os cenários, consideramos a utilização de processos para deixar a simulação mais realista: dois computadores tentando alocar o mesmo residente para o mesmo dia/turno/unidade. Com relação ao tratamento de concorrência, já foi citado que, no cenário otimista, o tratamento é simples por conta da definição da tabela, onde permite que haja apenas uma instância de dia/turno/unidade por residente. Já em relação ao cenário pessimista foi considerado que não há a propriedade da tabela Escala (o UNIQUE em dia/turno/unidade), com isto em mente, implementamos uma flag para a existência da instância dia/turno/unidade para aquele residente, caso a flag seja *true*, a inserção é abortada, caso contrário, é executada a inserção na tabela.

Front-end

A parte do front-end do projeto foi desenvolvida com **Streamlit**, um framework de código aberto desenvolvido para criar aplicações web interativas utilizando apenas Python. Assim, foram adicionadas 9 interfaces (ou abas) para as operações solicitadas pelo projeto:

- **Página Inicial:** Contém as informações gerais do projeto, incluindo a equipe responsável e do que a página front-end é composta.

A partir da página inicial, pode-se acessar as outras abas:

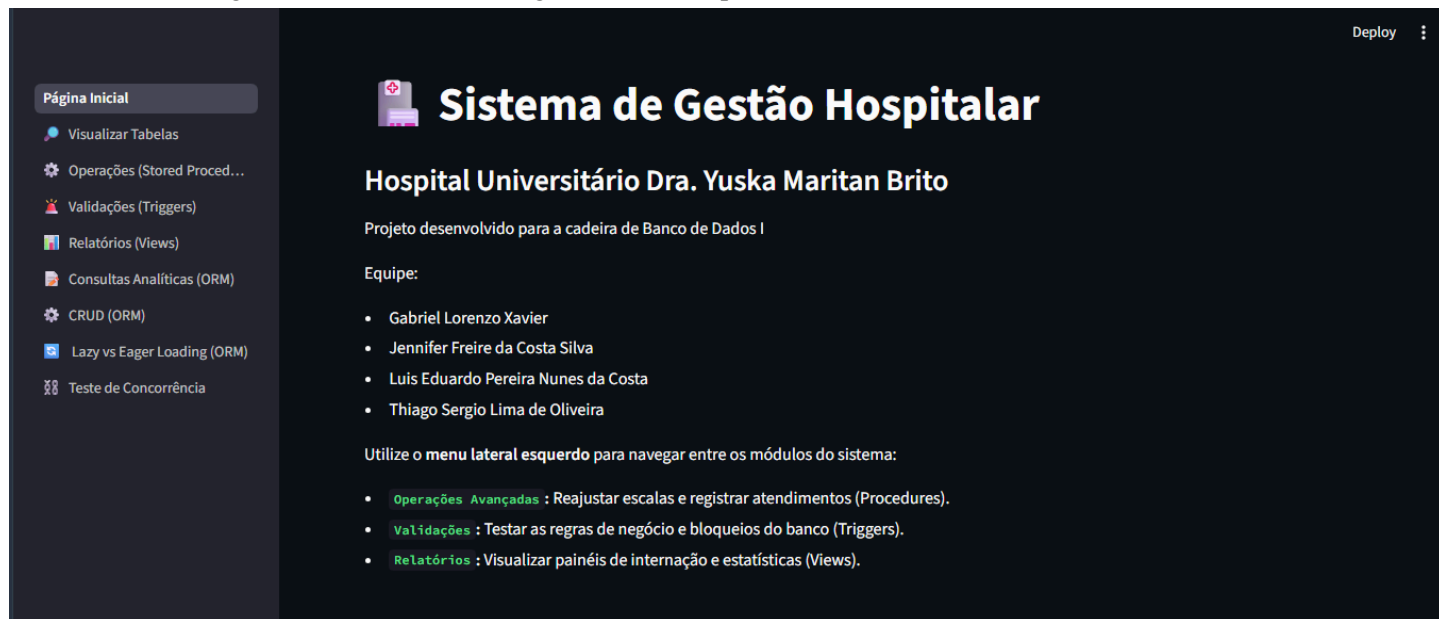
1. **Visualizar Tabelas:** Visualização dinâmica das tabelas existentes do banco de dados. Serve para facilitar a visualização dos dados que nelas estão contidos.
2. **Operações (Stored Procedures):** Contém 3 operações envolvendo as stored procedures, com dados personalizados e editáveis.
3. **Validações (Triggers):** Possui 3 operações para testar o funcionamento dos triggers que fazem parte da arquitetura do banco de dados.
4. **Relatórios (Views):** Nessa aba, são mostradas alguns scripts SQL que realizam a exibição das views definidas para o sistema.

Para a parte da ORM, foi realizada uma integração do código Python original com Streamlit, para permitir a visualização delas por meio do front-end:

5. **Consultas Analíticas (ORM):** Contém as consultas analíticas, definidas na etapa 1, reimplementadas usando a ORM SQLAlchemy.

6. **CRUD (ORM):** Esse arquivo possui as consultas básicas que compõem um CRUD definidas na etapa 1 e, também, reimplementadas com a ORM.
7. **Lazy vs Eager Loading (ORM):** Uma das demonstrações solicitadas pelo projeto envolvem a comparação entre o Lazy Loading e o Eager Loading. Assim, nessa aba, essa comparação pode ser vista pelo front-end, uma vez que a integração entre ORM e Streamlit foi feita.
8. **Teste de Concorrência:** A simulação de concorrência também feita com ORM está sendo mostrada no front-end. Nele, é mostrado dois cenários em que esse caso pode ocorrer: o cenário otimista e o cenário pessimista.

Por fim, segue uma *screenshot* da Página Inicial da aplicação:



Conclusão

As decisões de implementação adotadas buscaram centralizar regras de negócio no banco de dados, automatizar validações e simplificar consultas frequentes. A utilização de Stored Procedures, Triggers e Views permitiu atender aos requisitos propostos, mantendo a integridade dos dados, reduzindo a complexidade da aplicação e facilitando a manutenção do sistema.